

The MOO problem

Compute X_E and Y_N for the following MOO problem:

$$\begin{array}{llllll} \max z_1 & = & x_1 & + & x_2 & \\ \min z_2 & = & x_1 & + & 3x_2 & \\ s.t. & & 2x_1 & + & 3x_2 & \leq 30 \\ & & 3x_1 & + & 2x_2 & \leq 30 \\ & & x_1 & - & x_2 & \leq 5.5 \\ & & x_1 & , & x_2 & \in \mathbb{N} \end{array}$$

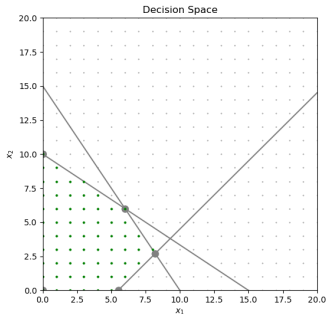
Example (1/4)



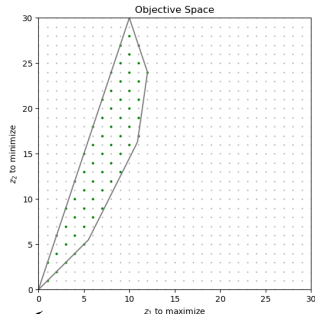
$$\begin{aligned}\max z_1 &= x_1 + x_2 \\ \min z_2 &= x_1 + 3x_2 \\ \text{s.t.} \quad 2x_1 + 3x_2 &\leq 30 \\ 3x_1 + 2x_2 &\leq 30 \\ x_1 - x_2 &\leq 5.5 \\ x_1, x_2 &\in \mathbb{N}\end{aligned}$$

```
julia> c1 = [1,1]
julia> c2 = [1,3]
julia> A = [2 3; 3 2; 1 -1]
julia> b = [30, 30, 5.5]
```

Decision space:



Objective space:





Coding the MOO problem with JuMP

```
julia> using JuMP, HiGHS
julia> import MultiObjectiveAlgorithms as MOA
```

```
julia> model = Model( )
julia> @variable(model, x1 ≥ 0, Int)
julia> @variable(model, x2 ≥ 0, Int)
julia> @expression(model, fct1, x1 + x2)           # to maximize
julia> @expression(model, fct2, x1 + 3 * x2)       # to minimize
julia> @objective(model, Max, [fct1, (-1) * fct2])
julia> @constraint(model, 2*x1 + 3*x2 ≤ 30)
julia> @constraint(model, 3*x1 + 2*x2 ≤ 30)
julia> @constraint(model, x1 - x2 ≤ 5.5)
```



Coding the MOO problem with JuMP

```
julia> using JuMP, HiGHS
julia> import MultiObjectiveAlgorithms as MOA
```

```
julia> model = Model( )
julia> @variable(model, x1 ≥ 0, Int)
julia> @variable(model, x2 ≥ 0, Int)
julia> @expression(model, fct1, x1 + x2)           # to maximize
julia> @expression(model, fct2, x1 + 3 * x2)       # to minimize
julia> @objective(model, Max, [fct1, (-1) * fct2])
julia> @constraint(model, 2*x1 + 3*x2 ≤ 30)
julia> @constraint(model, 3*x1 + 2*x2 ≤ 30)
julia> @constraint(model, x1 - x2 ≤ 5.5)
```



Setup the MIP solver: e.g. HiGHS

```
julia> set_optimizer(model, ()->MOA.Optimizer(HiGHS.Optimizer))
```

Setup the algorithm: ϵ -constraint; step=1 (default value)

```
julia> set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
```

Optimize the MOO problem

```
julia> optimize!(model)
```



Setup the MIP solver: e.g. HiGHS

```
julia> set_optimizer(model, ()->MOA.Optimizer(HiGHS.Optimizer))
```

Setup the algorithm: ϵ -constraint; step=1 (default value)

```
julia> set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
```

Optimize the MOO problem

```
julia> optimize!(model)
```



Setup the MIP solver: e.g. HiGHS

```
julia> set_optimizer(model, ()->MOA.Optimizer(HiGHS.Optimizer))
```

Setup the algorithm: ϵ -constraint; step=1 (default value)

```
julia> set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
```

Optimize the MOO problem

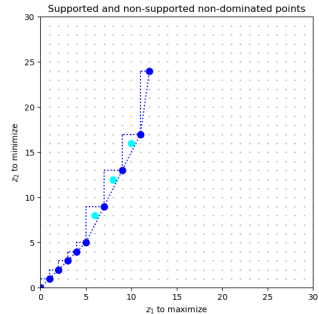
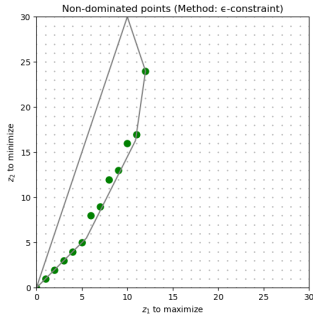
```
julia> optimize!(model)
```

Example (4/4)



Get X_E and Y_N

```
julia> for i in 1:result_count(model)
julia>     z1_opt = objective_value(model; result = i)[1]
julia>     z2_opt = -1 * objective_value(model; result = i)[2]
julia>     x1_opt = value(x1; result = i)
julia>     x2_opt = value(x2; result = i)
julia> end
```



Display of Y_N as a figure

```
julia> using Plots
julia> using LaTeXStrings
julia>
julia> Z1opt = [ ]; Z2opt = [ ]
julia> for i in 1:result_count(model)
julia>     push!(Z1opt, objective_value(model; result = i)[1])
julia>     push!(Z2opt, -1 * objective_value(model; result = i)[2])
julia> end
julia>
julia> plot(Z1opt, Z2opt,
julia>     title = "Nondominated points (e-constraint method)",
julia>     xlabel = L"$z_1$ to maximize",
julia>     xlabel = L"$z_2$ to minimize",
julia>     xlims = (0, 30),
julia>     ylims = (0, 30),
julia>     seriestype = :scatter,
julia>     marker = :circle,
julia>     markersize = 5,
julia>     color = :green,
julia>     legend = false
julia> )
```